

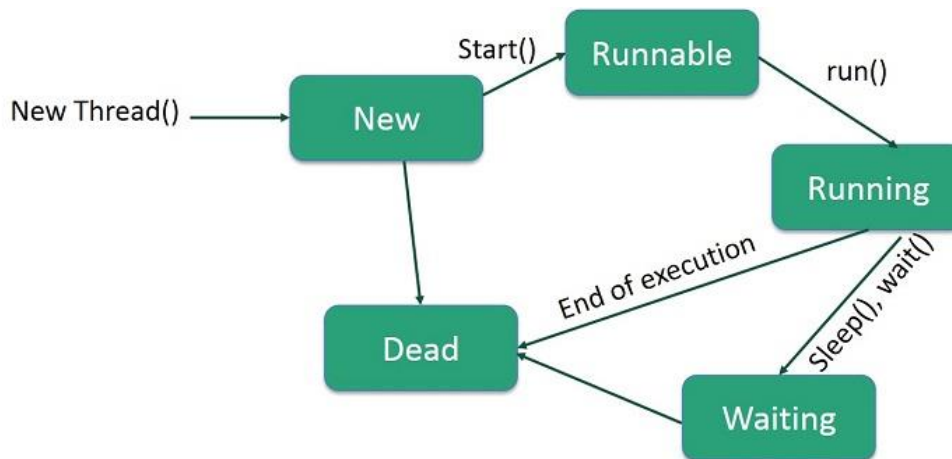
UNIT -III

❖ Life Cycle of a Thread in Java:

The life cycle of a thread in Java refers to the various states of a thread goes through. For example, a thread is born, started, runs, and then dies. Thread class defines the life cycle and various states of a thread.

Flow Chart of Java Thread Life Cycle

The following diagram shows the complete life cycle of a thread.



❖ Thread states

In Java, threads can exist in various states throughout their lifecycle. These states represent the different phases a thread goes through from its creation until its termination. The Thread class and related APIs provide methods to query and manage a thread's state. Here are the various thread states in Java:

1. New (NEW):

- When a thread is created using the new keyword or by instantiating the Thread class, it is in the "New" state. In this state, the thread has been created, but it has not yet started its execution.

2. Runnable (RUNNABLE):

- A thread enters the "Runnable" state when it is ready to run but has not been scheduled to run by the operating system yet. In this state, the thread can execute if the CPU scheduler assigns it a time slice

3. Waiting (WAITING):

- A thread enters the "Waiting" state when it is waiting for a specific condition to be met.

Unlike the "Blocked" state, a thread in the "Waiting" state is not actively trying to acquire a resource but is waiting for some external event to occur. Examples include waiting on an object's wait() method or a Thread.sleep() call.

4. Timed Waiting (TIMED_WAITING):

- Threads enter the "Timed Waiting" state when they are waiting for a specific condition for a limited time duration. Common examples include using Thread.sleep(timeout) or waiting for I/O with a timeout specified.

5. Terminated/Dead (TERMINATED):

- A thread enters the "Terminated" state when its run() method completes execution, or when an unhandled exception is thrown within the thread. A terminated thread cannot be restarted or resumed

Example:

```
package ThreadExample;
```

```
public class ThreadDemo extends Thread  
{
```

```
    private Thread t;  
    private String threadName;  
    ThreadDemo( String name)  
    {  
        threadName = name;  
        System.out.println("Thread: " + threadName + ", " + "State: New");  
    }  
    public void run() {  
        System.out.println("Thread: " + threadName + ", " + "State: Running");  
        for(int i = 4; i > 0; i--) {  
            System.out.println("Thread: " + threadName + ", " + i);  
        }  
        destroy();  
    }  
}
```

```
    void destroy() {  
        System.out.println("Thread: " + threadName + ", " + "State: Dead");  
    }  
}
```

```
public void StartThread () {  
    System.out.println("Thread: " + threadName + ", " + "State: Start");  
    if (t == null)  
    {  
        t = new Thread (this, threadName);  
        t.start ();  
    }  
}
```

```

    }
}

public static void main(String args[]) {
    ThreadDemo thread1 = new ThreadDemo( "Thread-1");
    ThreadDemo thread2 = new ThreadDemo( "Thread-2");
    thread1.StartThread();
    thread2. StartThread ();
}
}

```

❖ **Properties of the Thread Class:**

A thread is defined as the execution path of a program. Each thread defines a unique flow of control.

- **CurrentContext**
Gets the current context in which the thread is executing.
- **CurrentCulture**
Gets or sets the culture for the current thread.
- **CurrentPrinciple**
Gets or sets the thread's current principal (for role-based security).
- **CurrentThread**
Gets the currently running thread.
- **CurrentUICulture**
Gets or sets the current culture used by the Resource Manager to look up culture-specific resources at run-time.
- **ExecutionContext**
Gets an ExecutionContext object that contains information about the various contexts of the current thread.
- **IsAlive**
Gets a value indicating the execution status of the current thread.
- **IsBackground**
Gets or sets a value indicating whether or not a thread is a background thread.
- **IsThreadPoolThread**
Gets a value indicating whether or not a thread belongs to the managed thread pool.

- **ManagedThreadId**
Gets a unique identifier for the current managed thread.
- **Name**
Gets or sets the name of the thread.
- **Priority**
Gets or sets a value indicating the scheduling priority of a thread.

❖ **Setting Thread Priority**

You can set the priority of a thread using the `setPriority(int priority)` method, where `priority` is an integer between `Thread.MIN_PRIORITY` and `Thread.MAX_PRIORITY`.

Example:

```
public class ThreadPriorityExample {
    public static void main(String[] args) {
        // Creating threads with different priorities
        Thread t1 = new Thread() -> {
            for (int i = 1; i <= 5; i++) {
                System.out.println("Thread 1 executing");
            }
        });

        Thread t2 = new Thread() -> {
            for (int i = 1; i <= 5; i++) {
                System.out.println("Thread 2 executing");
            }
        });

        Thread t3 = new Thread() -> {
            for (int i = 1; i <= 5; i++) {
                System.out.println("Thread 3 executing");
            }
        });

        // Setting priorities
        t1.setPriority(Thread.MIN_PRIORITY); // Priority 1
        t2.setPriority(Thread.NORM_PRIORITY); // Priority 5 (default)
        t3.setPriority(Thread.MAX_PRIORITY); // Priority 10

        // Starting threads
        t1.start();
        t2.start();
        t3.start();
    }
}
```

❖ Methods in Threads and Runnable

In Java, the Thread class and the Runnable interface provide the foundation for creating and working with threads. These classes define various methods and mechanisms for managing threads and their execution.

Thread Class Methods:

1. start() Method:

The start() method is used to begin the execution of a thread. It internally calls the run() method.

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running.");
    }
}

public class ThreadExample {
    public static void main(String[] args) {
        MyThread myThread = new MyThread();
        myThread.start(); // Starts the thread's execution
    }
}
```

2. run() Method:

The run() method is the entry point for the code to be executed in the thread. You override this method when you extend the Thread class.

3. join() Method:

The join () method allows one thread to wait for another thread to complete its execution before continuing.

```
Thread thread1 = new Thread(() -> {
    System.out.println("Thread 1 is running.");
});
Thread thread2 = new Thread(() -> {
    System.out.println("Thread 2 is running.");
});
thread1.start();
thread1.join(); // Wait for thread1 to finish before proceeding
thread2.start();
```

4. sleep() Method:

The sleep() method pauses the execution of the current thread for a specified amount of time in milliseconds.

```
Thread thread = new Thread() -> {  
    try {  
        Thread.sleep(2000); // Sleep for 2 seconds  
        System.out.println("Thread resumed after sleep.");  
    } catch (InterruptedException e) {  
        e.printStackTrace();  
    }  
});  
thread.start();
```

5. getName() and setName() Methods:

getName() retrieves the name of the thread.

setName() sets the name of the thread.

```
Thread thread = new Thread() -> {  
    System.out.println("Thread name: " + Thread.currentThread().getName());  
});  
thread.setName("WorkerThread");  
thread.start();
```

❖ Synchronization in Java:

Java Synchronization is used to make sure by some synchronization method that only one thread can access the resource at a given point in time.

Types of Synchronization

There are two synchronizations in Java mentioned below:

1. Process Synchronization in Java

Process Synchronization is a technique used to coordinate the execution of multiple processes. It ensures that the shared resources are safe and in order.

2. Thread Synchronization in Java

Thread Synchronization is used to coordinate and ordering of the execution of the threads in a multi-threaded program. There are two types of thread synchronization are mentioned below:

- Mutual Exclusive

- Cooperation (Inter-thread communication in Java)

Mutual Exclusive

Mutual Exclusive helps keep threads from interfering with one another while sharing data. There are three types of Mutual Exclusive mentioned below:

- Synchronized method.
- Synchronized block.
- Static synchronization.

When Do We Use Synchronization?

We use synchronization in the following cases:

- Shared Resources Access: When multiple threads access and modify shared resources (e.g., variables, objects) simultaneously.
- Thread Interference: To prevent threads from interfering with each other's operations, leading to unexpected behavior.
- Data Consistency: To maintain data integrity by ensuring a thread completes its operation before other accesses the same resource.
- Critical Sections: When a part of the code (critical section) must be executed by only one thread at a time.

Example:

```
import java.io.*;
import java.util.*;

// A Class used to send a message
class Sender {
    public void send(String msg)
    {
        System.out.println("Sending\t" + msg);
        try {
            Thread.sleep(1000);
        }
        catch (Exception e) {
            System.out.println("Thread interrupted.");
        }
        System.out.println("\n" + msg + "Sent");
    }
}
```

```

// Class for send a message using Threads
class ThreadedSend extends Thread {
    private String msg;
    Sender sender;

    // Receives a message object and a string
    // message to be sent
    ThreadedSend(String m, Sender obj)
    {
        msg = m;
        sender = obj;
    }

    public void run()
    {
        // Only one thread can send a message
        // at a time.
        synchronized (sender)
        {
            // synchronizing the send object
            sender.send(msg);
        }
    }

    public static void main(String args[])
    {
        Sender send = new Sender();
        ThreadedSend S1 = new ThreadedSend(" Hi ", send);
        ThreadedSend S2 = new ThreadedSend(" Bye ", send);

        // Start two threads of ThreadedSend type
        S1.start();
        S2.start();

        // wait for threads to end
        try {
            S1.join();
            S2.join();
        }
        catch (Exception e){
            System.out.println("Interrupted")
        }
    }
}

```

❖ **Inter thread communication**

Inter-thread communication is important when you develop an application where two or more threads exchange some information. Inter-thread communication is achieved by using the wait(), notify(), and notifyAll() methods of the Object class.

There are three simple methods and a little trick which makes thread communication possible. All the three methods are listed below –

1. public void wait()

Causes the current thread to wait until another thread invokes the notify().

2. public void notify()

Wakes up a single thread that is waiting on this object's monitor.

3. public void notifyAll()

Wakes up all the threads that called wait() on the same object.

```
public class A {  
  
    int i;  
  
    boolean flag = false; // flag will be true when data production is over.  
  
    synchronized void deliver(int i)  
    {  
        if(flag)  
            try {  
                wait(); // Wait till a notification is received from Thread2. There will be no wastage of  
                time.  
            }  
            catch(InterruptedException ie) {  
                System.out.println(ie);  
            }  
        this.i = i;  
        flag = true; // When data production is over, it will store true into flag.  
        System.out.println("Data Delivered: " +i);  
    }  
}
```

```

        notify(); // When data production is over, it will notify Thread2 to use it.
    }
    synchronized int receive()
    {
        if(!flag)
        try {
            wait(); // Wait till a notification is received from Thread1.
        }
        catch(InterruptedException ie){
            System.out.println(ie);
        }
        System.out.println("Data Received: " + i);
        flag = false; // It will store false into flag when data is received.
        notify(); // When data received is over, it will notify Thread1 to produce next data.
        return i;
    }
}

public class Thread1 extends Thread
{
    A obj;

    Thread1(A obj)
    {
        this.obj = obj;
    }

    public void run()

```

```

    {
        for(int j = 1; j <= 5; j++){
            obj.deliver(j);
        }
    }
}

public class Thread2 extends Thread
{
    A obj;

    Thread2(A obj)
    {
        this.obj = obj;
    }

    public void run()
    {
        for(int k = 0; k <= 5; k++) {
            obj.receive();
        }
    }

    public static void main(String[] args)
    {
        A obj = new A(); // Creating an object of class A.

        // Creating two thread objects and pass reference variable obj as parameter to Thread1 and Thread2.

        Thread1 t1 = new Thread1(obj);

        Thread2 t2 = new Thread2(obj);

        // Run both threads.

```

```
t1.start();  
t2.start();  
  
}  
  
}  
  
}
```

1. In this example, wait() and notify() methods are called inside deliver() and receive() method. Both methods enable Thread1 to notify Thread2 after producing data and wait until Thread2 complete using.

2. In the same way, Thread2 after using data notifies Thread1 and waits until Thread1 produces and delivers the next data. Thus, the output comes in a synchronized form.

3. If the flag is true, Thread2 takes data from Thread1 and use it. When Thread1 is busy producing data, now and then Thread2 will check flag is true or not.

If the flag shows false, then Thread2 will wait for the object until it receives a notification from a notify() method.

When the data production is over, Thread1 will send a notification immediately to Thread2 to receive data. In this way, Thread1 and Thread2 communicate with each other efficiently.

❖ **Introduction to exception handling:**

The Exception Handling in Java is one of the powerful mechanism to handle the runtime errors so that the normal flow of the application can be maintained. Exception Handling is a mechanism to handle runtime errors such as ClassNotFoundException, IOException, SQLException, RemoteException, etc.

Key Concepts:

1. Exception:

- An exception is an abnormal event or error that occurs during program execution. Exceptions can be caused by various factors, such as invalid input, resource unavailability, or unexpected conditions.

2. Exception Handling:

- Exception handling is the process of dealing with exceptions in a structured and controlled manner. It allows a program to detect, handle, and recover from exceptional situations.

3. Try-Catch Blocks:

- In most programming languages, including Java, exception handling is implemented using try catch blocks.
- The code that may raise an exception is placed within the "try" block.
- If an exception occurs within the try block, it's caught and handled by the code in the "catch" block.

4. Throwing Exceptions:

- Exceptions can be explicitly thrown using the "throw" keyword. This allows developers to signal exceptional conditions programmatically.
- Throwing an exception transfers control to the nearest catch block that can handle that type of exception.

Basic Syntax in Java:

In Java, exceptions are handled using the try-catch block. Here's the basic syntax:

```
try
{
// Code that may throw an exception
}
catch (ExceptionType1 e1)
{
// Handle ExceptionType1
}
catch (ExceptionType2 e2)
{
// Handle ExceptionType2
}
finally
{
// Optional: Code that always executes, regardless of whether an exception occurred or not
}
```

- The try block contains code that might raise an exception.
- The catch blocks follow the try block and specify the exception type(s) they can handle.
- The finally block (optional) contains code that always executes, whether an exception occurred or not.

Example in Java:

```

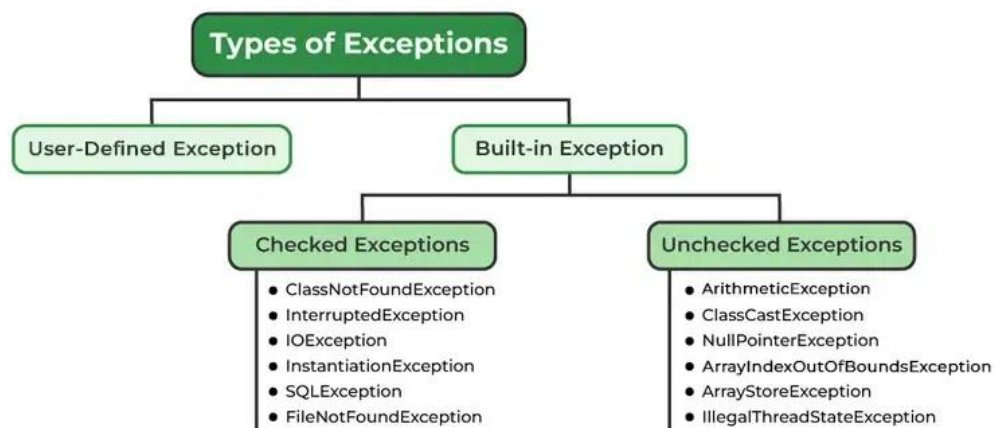
public class ExceptionHandlingExample
{
    public static void main(String[] args)
    {
        try {
            int result = divide(10, 0); // Attempt to divide by zero
            System.out.println("Result: " + result);
        }
        catch (ArithmeticException e)
        {
            System.err.println("Exception caught: " + e.getMessage());
        }
        finally
        {
            System.out.println("This is the 'finally' block.");
        }
    }

    public static int divide(int dividend, int divisor) {
        if (divisor == 0) {
            throw new ArithmeticException("Division by zero is not allowed.");
        }
        return dividend / divisor;
    }
}

```

❖ Types of Exceptions

Java defines several types of exceptions that relate to its various class libraries. Java also allows users to define their own exceptions.



Exceptions can be categorized in two ways:

1. **Built-in Exceptions**
2. **User-Defined Exceptions**

Let us discuss the above-defined listed exception that is as follows:

1. Built-in Exceptions

Built-in exceptions are the exceptions that are available in Java libraries. These exceptions are suitable to explain certain error situations.

a) **ArithmeticException:**

Thrown when an arithmetic operation (such as division) results in an exceptional condition, like division by zero.

```
public class ArithmeticExceptionExample {  
    public static void main(String[] args) {  
        int result = 10 / 0; // This will throw an ArithmeticException  
    }  
}
```

b) **NullPointerException:**

Thrown when you attempt to access an object or call a method on a null reference.

```
public class NullPointerExceptionExample  
{  
    public static void main(String[] args) {  
        String str = null;  
        int length = str.length(); // This will throw a NullPointerException  
    }  
}
```

c) **ArrayIndexOutOfBoundsException:**

Thrown when you attempt to access an array element with an invalid index.

```
public class ArrayIndexOutOfBoundsExceptionExample  
{  
    public static void main(String[] args) {  
        int[] array = { 1, 2, 3 };  
        int value = array[5]; // This will throw an ArrayIndexOutOfBoundsException  
    }  
}
```

d) **FileNotFoundException:**

Thrown when attempting to access a file that doesn't exist.

```
import java.io.File;  
import java.io.FileReader;  
import java.io.FileNotFoundException;  
public class FileNotFoundExceptionExample {  
    public static void main(String[] args) {  
        try {  
            FileReader fileReader = new FileReader(new File("nonexistent-file.txt"));
```

```

    } catch (FileNotFoundException e) {
        System.err.println("File not found: " + e.getMessage());
    }
}
}

```

2. User-Defined Exceptions:

Sometimes, the built-in exceptions in Java are not able to describe a certain situation. In such cases, users can also create exceptions, which are called 'user-defined Exceptions'.

The advantages of Exception Handling in Java are as follows:

1. Provision to Complete Program Execution
2. Easy Identification of Program Code and Error-Handling Code
3. Propagation of Errors
4. Meaningful Error Reporting
5. Identifying Error Types

1. Custom Checked Exception:

- A user-defined checked exception. You can create these when you want to handle specific application errors.

```

class CustomCheckedException extends Exception {
    public CustomCheckedException(String message) {
        super(message);
    }
}

public class CustomCheckedExceptionExample {
    public static void main(String[] args) {
        try {
            throw new CustomCheckedException("This is a custom checked exception.");
        } catch (CustomCheckedException e) {
            System.err.println("Custom checked exception caught: " + e.getMessage());
        }
    }
}

```

2. Custom Unchecked Exception:

- A user-defined unchecked exception. These are often used for programming errors or other exceptions that don't require explicit handling.

```

class CustomCheckedException extends Exception {

```



```

        public CustomCheckedException(String message) {
            super(message);
        }
    }
    public class CustomCheckedExceptionExample {
        public static void main(String[] args) {
            try {
                throw new CustomCheckedException("This is a custom checked exception.");
            } catch (CustomCheckedException e) {
                System.err.println("Custom checked exception caught: " + e.getMessage());
            }
        }
    }
}

```

Exception Handling Mechanisms:

1. Try-Catch Block

This is the most common way to handle exceptions. The try block contains code that may throw an exception, while the catch block handles it.

Example:

```

public class TryCatchExample {
    public static void main(String[] args) {
        try {
            int result = 10 / 0; // This will throw ArithmeticException
        } catch (ArithmeticException e) {
            System.out.println("Exception caught: Division by zero is not allowed.");
        }
    }
}

```

2. Try-Catch-Finally Block

The finally block executes code regardless of whether an exception occurred or not. It is often used for cleanup operations, like closing a file or releasing resources.

Example:

```

public class FinallyExample {
    public static void main(String[] args) {
        try {
            int[] numbers = {1, 2, 3};
            System.out.println(numbers[5]); // ArrayIndexOutOfBoundsException
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Exception caught: Index out of bounds.");
        } finally {
            System.out.println("Finally block executed.");
        }
    }
}

```

```
}  
}
```

3. Throwing an Exception

The throw keyword is used to explicitly throw an exception.

Example:

```
public class ThrowExample {  
    public static void main(String[] args) {  
        int age = 15;  
        if (age < 18) {  
            throw new IllegalArgumentException("Age must be 18 or above.");  
        }  
        System.out.println("Age is valid.");  
    }  
}
```

4. Declaring Exceptions with Throws

A method can declare exceptions it might throw using the throws keyword. The calling method must handle or propagate them.

Example:

```
public class ThrowsExample {  
    public static void main(String[] args) throws IOException {  
        readFile();  
    }  
  
    public static void readFile() throws IOException {  
        BufferedReader br = new BufferedReader(new FileReader("nonexistentfile.txt"));  
        System.out.println(br.readLine());  
    }  
}
```